A bank AI assistant that cannot leak what it never sees

Project briefing for the team and the judges. Everything built so far, why we built it, and what the numbers say.

The idea in four sentences

A bank wants staff to ask an AI questions about documents and customer cases. The risk is that the AI shows something the person is not allowed to see.

Our answer is to stop the problem early. Documents carry a clearance level (tier 1 to 3), and the database only hands over what the user's tier allows. Personal details like Emirates IDs, IBANs, phone numbers and names are replaced with tags like [IBAN] **before** any text reaches the AI.

Later, an attacker AI will keep probing the system so its guardrails get stronger by themselves, and every action goes into a log that cannot be edited without being noticed.

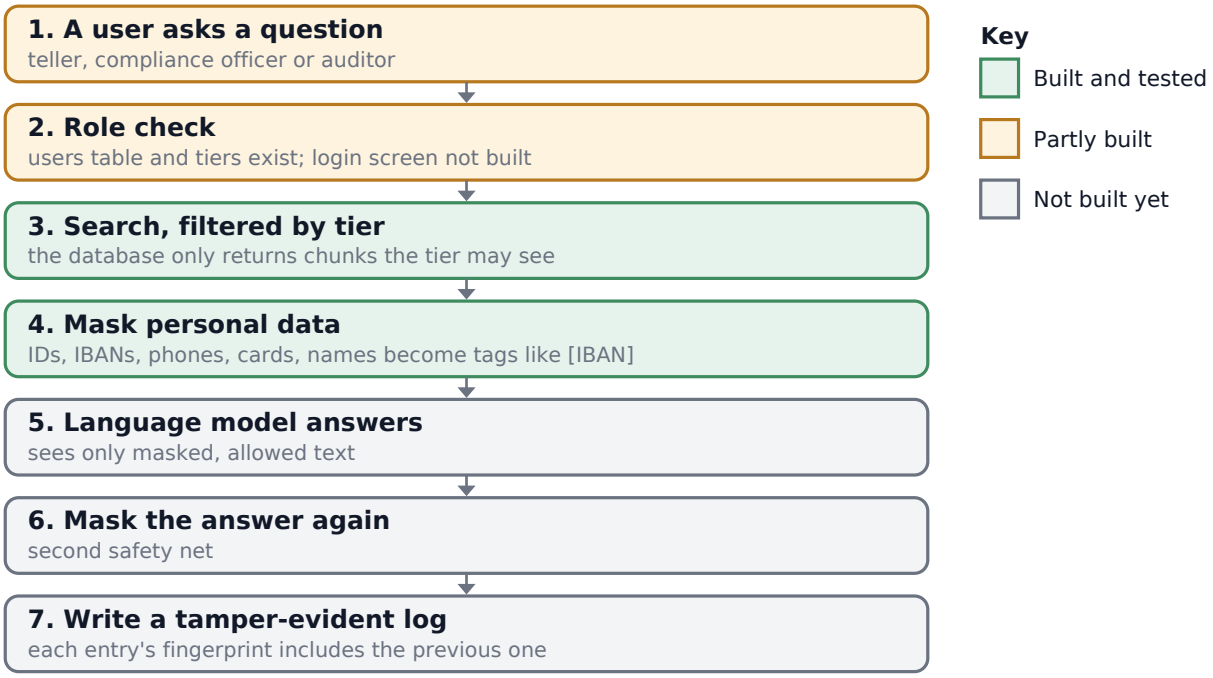
It all runs on an IBM LinuxONE mainframe, so the data stays on infrastructure the bank controls.

Where we are today

Part	Status	In one line
Server and database	**Done**	Mainframe server, PostgreSQL with vector search, notebooks, GitHub backup.
Data	**Done**	300 synthetic customers, 1,284 searchable chunks, 4 public datasets.
Tier-based search	**Done, tested**	Zero restricted results leaked in all 12 test runs.
Masker for IDs, cards, phones and more	**Done, measured**	96% to 100% found on most types on a fresh test set.
Masker for people's names	**Written, not yet scored**	Notebook 4b. Results come next.
AI answering step	Not built	Needs a free language model connected.
Tamper-evident log	Not built	Table exists; the fingerprint chain is still to write.
Attacker AI loop	Not built	Attack data downloaded; the loop is still to write.
Web demo with logins	Not built	Final step.

1. How the system works

This is the path one question takes. Green boxes are built and tested, amber is partly built, grey is still to do. Showing it this way is deliberate: it tells the judge exactly what is proven today.



Why two layers of protection?

- **Layer 1, the tier filter.** Restricted text is never even fetched. If the AI never receives it, the AI cannot repeat it.
- **Layer 2, the masker.** Even text a user is allowed to see can hold personal details. Those are hidden before the AI reads them.
- Either layer alone has gaps. Together they cover more.

Words you will need

Word	What it means in plain language
RAG	The AI looks up relevant documents first, then answers using them. Like an open-book exam.
Embedding	A list of numbers that captures the meaning of a text. Texts with similar meaning get similar numbers, so we can search by meaning.
Vector database	A database that can find the texts whose numbers are closest to a question's numbers. We use PostgreSQL with the pgvector add-on.
Chunk	A small piece of a long document (about a page or less), the unit we search over.
Tier	A clearance level from 1 (lowest) to 3 (highest). A user can only see chunks at or below their tier.
PII	Personally identifiable information: names, ID numbers, bank account numbers, phone numbers and similar.
Recall	Of all the sensitive values that were there, what share did we find and hide? High recall means few leaks.
Precision	Of the things we hid, what share really were sensitive? High precision means we do not hide harmless text.

Word	What it means in plain language
Benchmark	A public test set where the right answers are already marked, so we can score ourselves fairly.
Hash chain	Each log entry stores a fingerprint of itself plus the previous entry. Change one old entry and every later fingerprint breaks.

2. What we set up, and why

Piece	What we did	Why this choice
IBM LinuxONE Community Cloud	Registered, made an SSH key, deployed a free virtual server (Ubuntu, mainframe chip type s390x).	It is free, and it fits the story: sensitive bank data staying on a mainframe the bank controls.
Secure login from a Windows laptop	PowerShell and SSH with a private key file. A tunnel carries the notebook screen to the laptop browser.	Only the SSH door is open on the server. The tunnel avoids opening more doors.
JupyterLab inside tmux	Notebook environment on the server, kept alive inside a tmux session.	Notebooks show code, results and explanations together, and tmux keeps them running if the laptop disconnects.
Python environment and GitHub	A virtual environment and a private repository (bank-rag-guard) with a deploy key. Secrets and data files are excluded.	The server has no backups, so GitHub is our safety net. Excluding secrets prevents leaking keys.
PostgreSQL 16 + pgvector 0.6.0	Installed and tested vector search.	One database does both normal tables and meaning-based search, so the tier rule can sit inside the same query.
Jina embeddings (jina-embeddings-v3, 768 numbers)	Used through a web request to turn text into numbers.	It understands Arabic and English, and it has a free key without a card. Our first choice needed billing, so we switched.

Why no VS Code Remote?

The usual remote editing tool of VS Code does not run on this mainframe chip (s390x). So we use notebooks through a tunnel, plus Git. This was a real constraint we found and worked around, and it is good to mention to the judges.

3. The data, and why each dataset

The rule is simple: **no real customer data**. Everything is either made-up or already public. Each dataset has one clear job.

Dataset	What it is	Why we use it	Size we used
Synthetic customers (made with Python Faker)	Fake customers with English or Arabic names, Emirates IDs, UAE IBANs and phone numbers inside short case notes.	Gives bank-style data with no privacy risk. We know the true values, so we can check exactly what the masker missed. It is also our only Arabic test data.	300 notes (152 Arabic, 148 English)
Enron emails (public, from Carnegie Mellon)	Real business emails released during the Enron investigation.	Realistic, messy corporate text. We use it as the general document set and to check false alarms: ordinary emails should not trigger the UAE-specific rules.	1,500 emails (max. 25 per mailbox)
CUAD legal contracts (Atticus Project)	Commercial contracts with long, formal legal text.	Contracts are naturally sensitive, so they make a believable high-tier collection. They also test search over long documents.	About 26 MB of contract text; 384 chunks loaded
ai4privacy PII benchmark	English sentences where every piece of personal data is already marked.	An independent scoreboard. We did not write it, so scores on it are more honest than scores on our own data. It has no Arabic, which is why we made our own.	1,000 examples to build with, plus 1,000 unseen ones for the final test
deepset prompt-injections	Prompts that try to trick an AI, plus normal prompts, each labelled.	Seed material for the attacker AI in the self-healing loop. Downloaded, not used yet.	Train and test files

> **Be ready for this question: where do the tiers come from?**
>
> The tiers are **labels we assigned for testing**, not real bank clearances. Customer notes kept the tier generated with them. Emails were mostly tier 1, and contracts were set to tier 2 or 3. What the test proves is that the filter works, whatever the labels are.

We also planned Lakera Gandalf and HackAPrompt attack sets in the original idea. They are not downloaded yet, so we only claim the deepset set.

4. Notebook by notebook

Notebook 01: database setup

Goal: prove the server and database work, and create the three tables the project needs.

- Confirmed the machine really is the mainframe type (s390x) running Python 3.12.3.
- Connected to PostgreSQL 16.15 and checked that the vector add-on (pgvector 0.6.0) works with a tiny test: it correctly picked the closest of two vectors.
- Created three tables: `chunks` (text, tier 1 to 3, meaning-numbers), `users` (name, role, tier) and `audit_log` (who, what, details, previous fingerprint, own fingerprint).
- Added a fast index for the tier column and a fast similarity index for the meaning-numbers.

> **Say it simply**
> We built the filing cabinet first: one table for the document pieces, one for the users, one for the log. The log table exists, but the fingerprint logic that makes it tamper-evident is a later step.

Notebook 02: data preparation

Goal: create and collect all the data.

- Wrote small generators for Emirates IDs (format 784-YYYY-XXXXXXX-Z with a valid check digit) and UAE IBANs (23 characters with correct check digits).
- Generated 300 fake customer notes, half Arabic. About a third of the Arabic notes write numbers in Arabic-Indic digits, which is how people really write them. Tiers came out as 148, 100 and 52 notes for tiers 1, 2 and 3.
- Created three demo users: teller1 (tier 1), compliance1 (tier 3) and auditor1 (tier 1).
- Downloaded the public datasets. Two problems showed up and we fixed them (see section 6).

> **Say it simply**
> We made realistic fake customers so no real person is ever involved, and we kept the true answers so we can grade our own masking.

Notebook 03: loading the data and the access-control test

Goal: make everything searchable, and prove that a user can never get chunks above their tier.

- Cut everything into 1,284 chunks: 300 customer notes, 600 emails and 384 contract pieces.
- Turned each chunk into 768 numbers with Jina and stored it with its tier. A check showed the numbers capture meaning across languages: an English sentence and its Arabic translation scored 0.784 similarity, against 0.341 for an unrelated sentence.
- Wrote the search function with the rule written inside the database query: `WHERE tier <= user_tier`.
- Ran the **leakage test**: four sensitive questions asked as tier 1, tier 2 and tier 3 users. Each was run twice, once with the filter (the real system) and once without it as a control.

Chunks loaded	Tier 1	Tier 2	Tier 3	Total
Customer notes	148	100	52	300
Enron emails	351	190	59	600
Contract pieces	0	282	102	384
All	**499**	**572**	**213**	**1,284**

Leakage test result	Restricted chunks returned WITH the filter	Restricted chunks that WOULD have been returned WITHOUT it
Tier 1 user (4 questions, 40 results)	**0**	27 of 40 (about two thirds)
Tier 2 user (4 questions, 40 results)	**0**	5 of 40
Tier 3 user (4 questions, 40 results)	**0**	0 (tier 3 may see everything)

Say it simply

We asked the same questions with and without the safety rule. Without it, a junior user would have been handed restricted text in two out of every three results. With it, they got none. The control run matters: it shows the risk was real and the filter is what removed it.

Honest limits

Only four questions were tested. A larger test with tricky, adversarial questions is still to do. And this proves safety at the search step only, not yet for the whole pipeline with the AI.

Notebook 04: the masker for IDs, cards, phones and more

Goal: find personal data in text and replace it with a tag such as [IBAN] or [PHONE], in English and Arabic. We used fixed rules plus checks (for example, a card number must pass the standard Luhn check) instead of an AI model. The reason is privacy: asking an outside AI to find personal data would mean sending that data outside.

- **Arabic digits.** Before matching, Arabic-Indic digits are swapped for normal digits one-for-one. That way a number written in Arabic still gets caught, and character positions stay the same.
- **Types covered:** Emirates ID, IBAN (any country), phone numbers (UAE, international and local layouts), email, payment cards, card security codes, social security numbers, PINs, BIC bank codes, IP and MAC addresses, web links, crypto addresses and account numbers.
- **Method:** build a simple version, look at the real mistakes, improve, measure again. Every run was saved so the progress is visible.

Check 1: our own fake customers

The masker found **900 of 900** sensitive values (300 Emirates IDs, 300 IBANs, 300 phone numbers). That held for English and Arabic notes, and for Arabic digits and normal digits. But we wrote the data and the rules in the same format, so this is a sanity check and not an independent score.

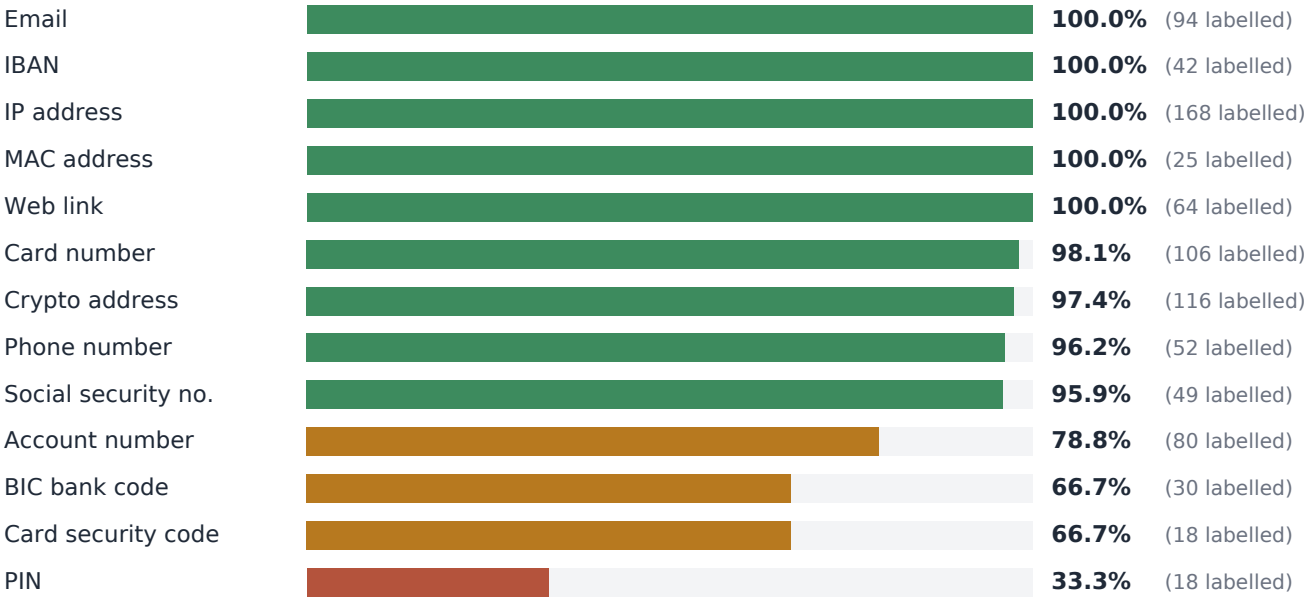
Check 2: how the benchmark scores improved

Type	First try (UAE-only rules)	After widening the rules
Phone numbers	1.5%	91.0%, then 96.2% on the unseen set
IBANs	0.0%	100%
Card numbers	not covered	17.5%, then 45.7%, then 96.9%
Account numbers	64.1%	71.9%

The first try failed on phones and IBANs because the benchmark uses other countries' formats and our first rules only knew UAE ones. That is a useful lesson to share: rules fitted to one region do not carry over.

Check 3: the final, fair test on 1,000 unseen examples

We tuned the rules while studying the first 1,000 benchmark examples, so scores on those are optimistic. To measure honestly, we took 1,000 different examples from much later in the same dataset, and ran the masker **without changing it**. These are the share of labelled values that were found (recall):



- **Precision was at least 97.7% for every type.** When the masker hid something, it was almost always right.
- **The weak types** (PIN, bank code, security code, account number) often appear as bare numbers or codes with no helpful word nearby, such as "patient 92445827 adverse reaction". A rule cannot safely tell these from harmless numbers, so we do not try. That is a real finding, and it is where a learned detector could add value later.
- The gap between the first set and the unseen set was about 4 points for phones and social security numbers, the two rules we shaped most by studying mistakes. That is the normal cost of tuning.

Check 4: false alarms on ordinary emails

We ran the masker over the 1,500 Enron emails. UAE-specific types (Emirates ID, IBAN) fired **zero times**, as they should. What did fire were real things: email addresses in 28.8% of emails, phone numbers in signatures, web links, IP addresses in mail headers, and a handful of real card numbers in travel invoices (6 matches after one false alarm was fixed). We read a random sample of 15 phone matches and all 15 were real phone numbers or fax numbers.

Notebook 4b: the name masker (written, not yet scored)

Goal: hide people's names in English and Arabic, which is the most important piece for a bank because names appear in almost every case note.

- Uses cue words (titles like Mr and Dr, greetings, sign-offs, words like "patient" or "customer", and Arabic cues for client, Mr, Dr and similar), plus name lists from the Faker library.
- Runs entirely on our server. Again we avoid an outside AI so raw names are never sent out.
- Keeps titles visible and hides only the name, for example "Dear [NAME]".
- Combined with the earlier masker in a single file, so identifiers are never cut in half by a name tag.

Plain status

This code is written but we have not run it and scored it yet, so we make **no accuracy claim** about names. Arabic names will be tested on our own fake notes, which share name lists with the masker, so those results will look better than reality. A better test is to have teammates each write ten natural Arabic sentences that mention people.

5. Problems we hit and how we solved them

Judges like seeing real engineering. Each of these is true and each cost us time.

Problem	What happened	What we did
VS Code remote editing does not work	The tool has no version for this mainframe chip.	Switched to JupyterLab through a secure tunnel, and Git for syncing.
Some Python packages would not install	No ready-made builds exist for the mainframe chip, so pip tried to compile them. One needed extra tools, and the data library needed a compiler we did not have.	Installed the missing tools for one package. For the other, dropped the library and downloaded data with plain web requests.
Google's AI key was refused	New-style keys returned an authentication error, and the setup pushed us toward billing.	Did not enable billing. Switched embeddings to Jina, which is free without a card and handles Arabic.
One Enron source disappeared	The Hugging Face copy returned 404 (not found).	Streamed the original archive from Carnegie Mellon and stopped after 1,500 emails.
First masker rules scored badly	Phones 1.5% and IBANs 0% on the benchmark.	Looked at the real misses, generalised the rules, re-measured, and kept every run.
A false alarm: a record ID read as a card	A long number inside a web link was masked as a card.	Added rules for numbers after an equals sign and numbers with many leading zeros.
Risk of fooling ourselves	We tuned rules on the same data we scored on.	Scored on a separate unseen slice and reported both.

6. Honest limits, and what is still to build

Say these before a judge asks

- **Not finished:** the AI answering step, the tamper-evident log, the attacker loop and the web demo. So "restricted data never enters the AI" is proven at the search step only so far.
- **The leakage test is small:** four questions. It needs many more, including tricky ones.
- **The benchmark is synthetic and English only.** Real-world accuracy would be lower, and Arabic has no independent test yet.
- **Rules have a ceiling:** values with no distinctive shape and no helpful nearby word cannot be caught safely.
- **Data:** the Enron emails are real public messages. Everything else is made-up or public. Only synthetic and public text was sent to the embedding service. A real bank would run that model inside its own network.
- **Tiers are test labels,** not real clearances.
- **No speed numbers:** we have not measured performance on the mainframe, so we do not claim any.

Still to build, in order

Step	What	How we will know it works
1	Score the name masker, then improve it from its mistakes	Recall and precision on the benchmark, plus a fresh unseen slice
2	Tamper-evident log	Edit one row on purpose and show that the chain check fails
3	AI answering step with a free model	Teller and compliance officer get different answers to the same question
4	Attacker loop that updates guardrails	Share of attacks blocked rises over rounds
5	Web demo with logins	A judge can try it live
6	Final evaluation write-up	One table of every number, with limits stated

7. Questions a judge may ask

Question	A good answer
Why filter in the database and not in the prompt?	Once restricted text is in the prompt, the AI can repeat it. Filtering before retrieval means it never gets there.
Why not use an AI to find the personal data?	That would send the raw personal data to a third party, which defeats the purpose. Local rules keep it in-house, at the cost of missing values that have no clue around them.
How do you know it works?	We measured recall and precision on a benchmark we did not write, and re-measured on 1,000 examples we never tuned on. We also ran the leakage test with a control that shows what the filter prevented.
Is any real data used?	Customer records are made-up. The contracts and Enron emails are public datasets.
Why a mainframe?	The data stays on infrastructure the bank controls, and the system runs on LinuxONE's architecture. We have not measured performance, so we do not claim speed.
What is your biggest weakness?	The pipeline is not finished yet, and some value types such as bare PINs cannot be caught by rules. We say so, and we show where a learned detector could help.
What would you do with real data?	Host the embedding model inside the bank's network, use real clearance levels from the bank's access system, and test with a bigger adversarial question set.

The 60-second version to say out loud

We built a bank AI assistant where restricted data never reaches the model. Documents carry clearance tiers, and the database filters by tier before anything is fetched, so a teller's question cannot even see compliance-level text. We tested it: with the filter there were zero restricted results, and without it a junior user would have received restricted content in two thirds of results.

Before text reaches the model, a masker replaces IDs, IBANs, phone numbers and cards with tags, including numbers written in Arabic digits. On 1,000 benchmark examples we never tuned on, it found 96% to 100% of emails, IBANs, phones, cards, IP addresses and social security numbers. We also report the weaker types openly. Names are the next step. Everything runs on an IBM LinuxONE mainframe. Still to come: a tamper-evident log and an attacker AI that hardens the guardrails.